# Multi-Source Transfer Learning for Design Technology Co-Optimization

Jakang Lee, Jaeseung Lee, Seonghyeon Park and Seokhyeong Kang
*Department of Electrical Engineering*,
*Pohang University of Science and Technology, Pohang, South Korea*
{wkrkd95, jae2seung, seonghyeon98, shkang}@postech.ac.kr

***Abstract*—In advanced technology nodes, pitch scaling have not kept up with the Moore's Law. To continue progression, the design technology co-optimization (DTCO) has been proposed. However, implementing DTCO requires significant time cost and resources due to iterative trials. In addition, optimal design and technology option depend on each design, thus it should start from scratch whenever the target design changes. We present a DTCO framework based on Bayesian optimization that efficiently explores design feedback for optimization. In addition, our framework incorporates a multi-source transfer Gaussian process (MTGP) that ensures robust optimization even for unseen designs. MTGP significantly improves prediction and generalization performance by integrating multiple single source transfer Gaussian processes. Our framework, on average, reduced the mean absolute error of power and area by 47.3% and 24.1%, respectively, and power and area by 37.3% and 19.9%, respectively, compared to the reference, in 7nm technology nodes.**

***Index Terms*—Design Technology Co-Optimization, Bayesian Optimization, Gaussian Process, Transfer Learning**

## I. Introduction

In recent decades, improvements in CMOS technology have largely been achieved through device scaling. However, as technology nodes shrink to sub-10nm, achieving the same level of power, performance, and area (PPA) improvements through scaling has become increasingly challenging. To overcome this issue, a new approach called Design Technology Co-Optimization (DTCO) has been introduced.

DTCO is an optimization methodology that encompasses both design and technology. The general flow of DTCO is depicted in Fig. 1, which highlights how foundries can customize technology to meet specific design requirements. This approach provides chip designers with an assortment of process design kits (PDK) that are optimized for PPA [1], [2]. As technology continues to develop, it becomes increasingly difficult to improve PPA, highlighting the crucial role that DTCO plays in advanced nodes [2].

Continuing with this trend, several studies have recently introduced DTCO technologies to enhance the design quality in advanced nodes [3]–[5]. All of these studies achieved significant design quality improvement by modifying the standard cell layout and various design and technology options. Nevertheless, these studies did not address the issue of design feedback that explores the optimal technology option again based on the PPA of the design. Hence, to achieve optimization using these approaches, the designer should evaluate the PPA of the design and select a suitable solution based on their expertise, making the optimization performance heavily reliant on the designer's knowledge. Therefore, there is an urgent need for a method that can effectively integrate design feedback into DTCO to promote efficient optimization.

The problem of reflecting design feedback effectively is one of the design space exploration problems that selects the design and technology option that is predicted to be optimal. In the electronic design automation (EDA), several recent works have proposed techniques for efficiently exploring large design spaces [6]–[9]. These works demonstrate the effectiveness of Bayesian optimization for exploring vast design spaces. However, a major limitation of these methods is their inability to reuse data since PPA to be predicted are highly dependent on the target design. Consequently, it is challenging to apply these methods when sufficient data is not available due to design time constraints. Therefore, there is a growing need for techniques that enable the reuse of existing design data and facilitate the training of predictive models for new designs.

979-8-3503-1175-4/23/$31.00 ©2023 IEEE

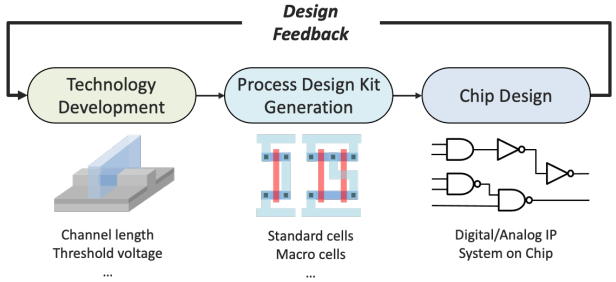


Fig. 1. Common DTCO flow

Transfer learning emerged as an optimal solution for reusing existing data [10], [11]. This approach uses source design data to construct a predictive model for target design data. Source design data means the PPA data of the previous design, and target design data means the PPA data of the design to be optimized. In general, transfer learning proceeds using a large amount of source design data compared to target design data. The aforementioned studies [10], [11] have demonstrated that transfer learning can enhance the accuracy of predictions and enable more efficient optimization search, even when there is a scarcity of target design data. However, one of the limitations of this method is that it can only utilize a single design as the source design data, thus it cannot incorporate multiple source design. As the performance of the target model is highly dependent by the specific source design used for transfer, this can lead to considerable variability in performance, which, in turn, results in a significant reduction in the generalization performance of the model.

In this study, we introduce a multi-source transfer Gaussian process (MTGP) based DTCO framework. Our framework can effectively explore the near-optimal design and technology options within a large design space. We incorporated Bayesian optimization to efficiently reflect design feedback, which is widely used due to its strong optimization performance, even with noise like the results of EDA tools [6], [7]. Furthermore, we utilized the GP as a surrogate model for Bayesian optimization and applied the MTGP method to overcome the drawbacks of the single-source transfer Gaussian process (STGP) approach [10]. MTGP is a method that utilizes data from multiple source designs for transfer to improve prediction performance and reduce generalization errors in target design. The major contributions of this study is summarized as follows.

- We propose a framework exploiting MTGP based Bayesian optimization to efficiently reflect design feedback in DTCO flow, thereby reducing the number of optimization iterations.
- We demonstrate that small designs can be utilized to improve the PPA predictive model performance of large designs, indicating the possibility of scalability for larger designs.

- By transferring data from several designs, the predictive performance dependency on source design is reduced, resulting in robust optimization performance.

## II. Preliminaries

### A. DTCO knobs

At the system level, achieving a balance between performance and power consumption is a challenge, as it is difficult to achieve both high performance and low power simultaneously. Therefore, designers need to prioritize and optimize designs based on their intended purpose. Our approach focuses on reducing power and area, while ensuring that the design meets the required timing constraints. To achieve this, we adjust the three key elements of DTCO knobs, which are described in detail in [1].

*1) Supply Voltage:* The power consumption is primarily dependent on the supply voltage. Specifically, the switching power is proportional to the square of the supply voltage, which implies that scaling the supply voltage can significantly reduce dynamic power consumption. To achieve energy efficiency, modern designs often employ voltage scaling and multiple supply voltages [12]. However, reducing the supply voltage negatively impact the timing performance of the device. Hence, setting an appropriate supply voltage that meets the target specifications is crucial to maintaining the performance.

*2) Threshold Voltage:* This is related to the sub-threshold leakage power. As the threshold voltage increases, the leakage power decreases exponentially. However, as with the supply voltage, increasing the threshold voltage degrades the timing performance of the device. Therefore, achieving an optimal balance between power consumption and performance is crucial. To balance leakage power and timing performance, modern chip designs use Multi-$V_{th}$ technique to selectively place the low-$V_{th}$ in the timing-critical path and selectively place the high-$V_{th}$ cell in the non-critical path.

*3) Standard Cell Height:* The height of a standard cell is often expressed as a horizontal routing track. Figure 2 shows cells with 7.5 and 6 routing tracks, respectively. Here, the two cells differ in the number of fins and their height. Increasing the number of tracks improves the device's timing performance, but also increases power consumption and area. Therefore, high-track cells are used where high performance is required, and low-track cells are used where low power and high density are required.

The three factors discussed earlier have a significant impact on power consumption, timing performance, and chip area. Moreover, the complex interdependency among these factors requires designers to search for an optimal balance between them by considering various design and technology options carefully.

### B. Threshold Voltage Engineering

The threshold voltage formula is given as:

$$V_{th0} = V_{fb} + \phi_S + \left.\frac{\sqrt{qN_a 2\epsilon_s \phi_S}}{C_{ox}}\right|_{\phi_S = 2\phi_B}, \tag{1}$$

$$V_{fb} = \Phi_G - \Phi_S - \frac{Q_{ox}}{C_{ox}}, \quad C_{ox} = \frac{\epsilon_{ox}}{T_{ox}}, \tag{2}$$

where $V_{th0}$ is a threshold voltage, $\phi_S$ is the surface potential of the channel, $q$ is the charge of the electron, $N_a$ is acceptor concentration of the channel, $\epsilon_s$ is the dielectric constant of semiconductor, $C_{ox}$ is the gate capacitance, $V_{fb}$ is a work function difference between gate electrode and the silicon.

According to the equations (1)-(2), there are four methods for modifying the threshold voltage of a transistor. Traditional methods that control the doping concentration of the channel ($N_a$) are not suitable for advanced nodes due to the random fluctuations of dopants. Similarly, modifying the gate dielectric constant ($\epsilon_{ox}$) is difficult to implement at advanced nodes that use metal gates, as only metal and silicon-compatible materials can be used as dielectrics.

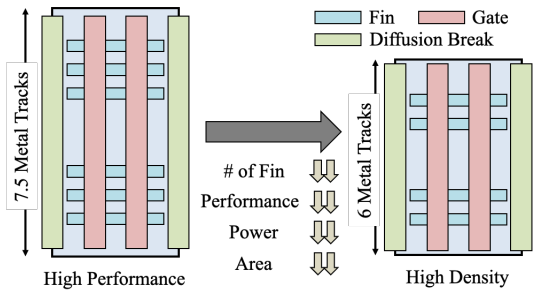


Fig. 2. Standard cell height

Therefore, dielectric thickness tuning ($T_{ox}$) and gate work function ($\Phi_G$) engineering are commonly used techniques for threshold voltage control.

Increasing the thickness of the dielectric raises the threshold voltage and reduces the gate oxide tunneling current, which greatly reduces leakage power. However, if the thickness is reduced too much, the leakage power increases excessively. Therefore, this approach is mainly used for manufacturing high $V_{th}$ cells. Gate work function engineering is considered the most effective approach for providing multiple threshold voltages.

In light of the aforementioned information, we adopt gate work function engineering to fine-tune the threshold voltage. By directly manipulating the technology, this method also enables us to consider potential side effects that may arise from changes to the process parameters, such as the gate fringing capacitance. Consequently, the reliability of our study can be significantly enhanced.

### C. Gaussian Process

The field of EDA often employs heuristic algorithms to generate results, which can be challenging to define and may contain noise. To address this issue, the EDA industry has undertaken numerous studies utilizing GP to model noisy black box problems. GP is a powerful technique estimates both predicted values and their corresponding uncertainties using the covariance matrix. The posterior mean and variance of the GP are calculated using conditional probabilities in the following manner:

$$\mu_* = k(X_*, X)^\top (k(X, X) + \sigma^2 I)^{-1} Y,$$

$$\sigma_*^2 = k(X_*, X_*) - k(X_*, X)^\top (k(X, X) + \sigma^2 I)^{-1} k(X_*, X), \tag{3}$$

where the new input values for a prediction are denoted by $X_*$, while the previous inputs are represented by $X$. Additionally, the predicted value of the objective function for the new input is denoted by $\mu_*$, while $\sigma_*^2$ denotes the uncertainty of the prediction. The previous output values are denoted by $Y$, while $k(X, X)$ represents the covariance matrix. In most cases, the covariance matrix is replaced with a kernel function.

A widely employed approach for training GP models is to identify a suitable kernel function and associated parameters that accurately represent the given data. To accomplish this, maximum likelihood estimation is commonly utilized. The objective of maximum likelihood estimation is to find a kernel function that maximizes the likelihood of observed data given a GP model. Based on this, kernel parameters and data noise are adjusted to achieve the best fit for the model. The kernel function is of paramount importance in GP learning as it determines the shape and continuity of the GP. The radial basis function (RBF) is frequently used as a kernel function in GP learning because of its smoothness and infinite differentiability, making it a popular choice.

### D. Transfer Kernel

GP is a powerful modeling technique, but efficiency is reduced because a new model must be created every time the target design changes. To overcome this limitation, transfer learning has emerged as a promising approach. In this section, we introduce the transfer kernel method, which is applicable to GP. This method learns the similarity between different designs as a kernel parameter and utilizes data from the different design to learn the GP of the target design. The modified kernel function for transfer learning can be expressed as:

$$K_{nm}(x_n, x_m) = \begin{cases} \lambda k(x_n, x_m), x_n \in S \ \& \ x_m \in T, \\ k(x_n, x_m), otherwise, \end{cases} \quad (4)$$

where $S$ is one source design, $\lambda$ is the coefficient for the relatedness of the source and target designs, and $k(x, x')$ is a kernel function.

Geng *et al.* [10] constrained $\lambda$ within the bounds of -1 to 1 to facilitate acquisition of both positive and negative correlations. Specifically, when $|\lambda| = 1$, $\lambda k(x, x')$ indicates that the source and target designs are closely related, and all source design data is transferred to the target design. Conversely, when $\lambda = 0$, $\lambda k(x, x')$ implies that only the target design data is used for training, as the source and target designs are entirely dissimilar. According to Equation (4), the covariance matrix of the target design's GP changes as:

$$\tilde{\mathbf{K}} = \begin{pmatrix} K_{SS} & \lambda K_{ST} \\ \lambda K_{TS} & K_{TT} \end{pmatrix}, \quad (5)$$

where $K_{SS}$ refers to the covariance matrix generated exclusively from the source design data, while $K_{TT}$ denotes the covariance matrix produced solely from the target design data. Furthermore, $\lambda K_{ST}$(=$\lambda K_{TS}^{\top}$) is the covariance matrix constructed to capture the resemblance between the source and target designs. Utilizing the covariance matrix at hand, Equation (3) may be adapted as follows.

$$\mu_* = \mathbf{k}_*(X_*, X)^{\top}(\tilde{\mathbf{K}}(X, X) + \mathbf{\Sigma})^{-1}Y,$$
$$\sigma_*^2 = k(X_*, X_*) - \mathbf{k}_*(X_*, X)^{\top}(\tilde{\mathbf{K}}(X, X) + \mathbf{\Sigma})^{-1}\mathbf{k}_*(X_*, X), \quad (6)$$

where the matrix $\mathbf{\Sigma}$ incorporates the noise present in the source and target design data, and the matrix $\mathbf{k}_*$ is used to estimate the existing data by dividing it into source and target components.

$$\mathbf{\Sigma} = \begin{pmatrix} \sigma_s^2 \mathrm{I_s} & 0 \\ 0 & \sigma_t^2 \mathrm{I_t}, \end{pmatrix} \quad \mathbf{k}_*(X_*, X) = \begin{pmatrix} \lambda k(X_*, X), X \in S \\ k(X_*, X), X \in T \end{pmatrix} \quad (7)$$

Through Equations (6)-(7), the posterior distribution of the target design's GP given the source design data can be obtained. It could enhance the performance of the model by leveraging correlations ($\lambda$) between the source and target design datasets. However, this method may increase the likelihood of overfitting the target GP with respect to specific source design data because of limited capacity.

## III. Proposed Method

### A. Overall Framework

The objective of this research is to explore the design and technology options that can achieve power and area optimization while meeting the timing constraints. The iterative process from RTL to GDS incurs significant costs and time consumption. To mitigate this challenge, we concentrate on cost-effective optimization following logic synthesis. Our proposed framework encompasses five major steps, including the selection of the design and technology options, generation of PDK, logic synthesis, establishment of multi-source transfer GP (MTGP), measurement of PPA and utilization of Bayesian optimization for selecting the optimal design and technology options. Fig. 3 shows the overall framework.

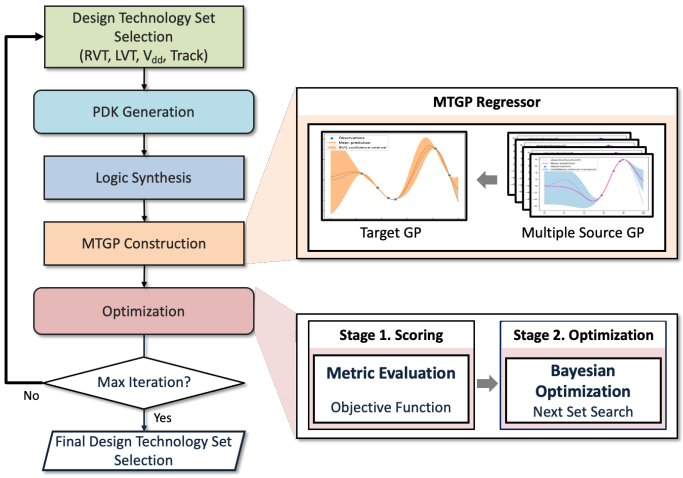


Fig. 3. Overall framework

### B. Multi-source Transfer Gaussian Process

We introduce MTGP to overcome the overfitting problem due to the capacity limitations of STGP mentioned in Section II-D. MTGP has been proposed to reduce generalization errors in the target design by leveraging knowledge from multiple source designs [15]. The formula for MTGP is:

$$k_\lambda(x, x') = \begin{cases} \lambda_{D_i, D_j} k(x, x'), x \in D_i \ \& \ x' \in D_j, i \neq j \\ k(x, x'), x \ \& \ x' \ in \ the \ same \ domain \end{cases}, \quad (8)$$

where $D$ is defined as a set containing both source and target designs, while kernel function is represented by $k(x, x')$. As explained in II-D, each domain is assigned a similarity metric $\lambda_{D_i, D_j}$, which is re-parameterized as $\lambda_{D_i, D_j} = \alpha_{D_i}\alpha_{D_j}$. The parameter $\alpha_{D_i}$ represents the similarity between the $i$-th domain and the target domain. As MTGP is trained, $\lambda_{T,S}$ and $\lambda_{S,S}$ could gradually acquire information about the relationship between the target and source designs and the source and source designs, respectively. The covariance matrix employed in this method is expressed as follows.

$$\tilde{\mathbf{K}} = \begin{pmatrix} K_{S_1,S_1} & \lambda_{S_1,S_2} K_{S_1,S_2} & \dots & \lambda_{S_1,T} K_{S_1,T} \\ \lambda_{S_2,S_1} K_{S_2,S_1} & K_{S_2,S_2} & \dots & \lambda_{S_2,T} K_{S_2,T} \\ \dots & \dots & \dots & \dots \\ \lambda_{T,S_1} K_{T,S_1} & \lambda_{T,S_2} K_{T,S_2} & \dots & K_{T,T} \end{pmatrix} \quad (9)$$

MTGP can effectively consider similarity between all source and target designs by Equation (9) and has improved generalization error compared to STGP. The process of learning MTGP is accomplished by utilizing maximum likelihood estimation, a method similar to that of conventional GP. However, since the source design data induces modifications in the kernel, adjustments are necessary. The adapted log-likelihood function for MTGP is presented below:

$$L_\Theta = \log p(Y_T | X_T, X_S, Y_S; \Theta)$$
$$= -\frac{1}{2}(\log |{\sigma_*}^2| + (Y_T - \mu_*)^{\top} {\sigma_*}^{-2} (Y_T - \mu_*) + N \log(2\pi)), \quad (10)$$

where the values of $\mu_*$ and $\sigma_*^2$ can be obtained by extending the source part in Equation (6), where $N$ represents the number of data used for training. The set of parameters that need to be learned is represented by $\Theta$, which includes kernel parameters, data noise ($\sigma_{s1}^2$, ..., $\sigma_{s2}^2$, $\sigma_t^2$), and the similarity of the domain ($\alpha_D$).

The benefits of MTGP can be summarized as follows.

- The utilization of multiple sources for training in MTGP results in an enhancement of its generalization performance.

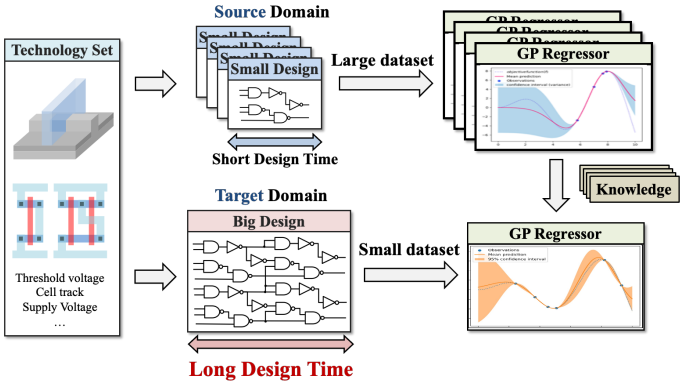


Fig. 4. The necessity of MTGP for efficient DTCO: multiple small design designs can be reused for larger design prediction models.

- MTGP can enhance the accuracy of the model in the target design even in situations where there is a limited amount of target design data available.
- MTGP achieve greater expressiveness by taking into account not only the similarity between the source and target designs but also the similarity between different source designs.

MTGP can leverage these capabilities to enhance the efficiency of DTCO by improving its optimization performance.

### C. Cost Function

Our objective is to minimize both power and area while meeting the timing constraints. Due to the inherent trade-offs among PPA, a flexible black-box optimization approach is necessary. To address this, we referred to [9] to construct a weighted sum cost function that accounts for multiple metrics. Given that each metric has distinct units, we normalized each metric to a value determined using the reference foundry-set provided.

$$P_{nor} = \frac{P_{cur} - P_{ref}}{P_{ref}}, \quad A_{nor} = \frac{A_{cur} - A_{ref}}{A_{ref}}, \quad S_{nor} = \frac{S_{cur}}{P_{clock}}, \tag{11}$$

where the normalized power, area, and worst negative slack (WNS) are denoted as $P_{nor}$, $A_{nor}$, and $S_{nor}$, respectively. The power, area, and WNS of the current design using the modified design and technology options are denoted as $P_{cur}$, $A_{cur}$, and $S_{cur}$, while the power and area obtained from the reference foundry-set are represented by $P_{ref}$ and $A_{ref}$, respectively. The clock period used for that design is denoted as $P_{clock}$.

We then utilized a single-objective Bayesian optimization framework to optimize the weighted-sum cost function. The specific form of the cost function we employed is as follows:

$$f = \begin{cases} \alpha \cdot P_{nor} + \beta \cdot A_{nor}, & S_{cur} > 0 \\ \alpha \cdot P_{nor} + \beta \cdot A_{nor} - \gamma \cdot S_{nor}, & S_{cur} < 0 \end{cases}, \tag{12}$$

where the weights for power and area are denoted by $\alpha$ and $\beta$, respectively. Furthermore, the penalty weight for WNS is represented by $\gamma$, since power and area improvements are performed under the assumption that timing violations are avoided. Consequently, a penalty is imposed when the timing constraints are not met. The designer can adjust these values flexibly based on the optimization goals of the framework.

### D. Bayesian Optimization

Bayesian optimization is a promising approach for optimizing black-box problems, which consists of two main components: a surrogate model and an acquisition function. The surrogate model is utilized to estimate the cost function of a black box, while the acquisition function is responsible for searching for a point where the cost function is expected to be minimized, based on the surrogate model. At this stage, it is crucial to balance the trade-off between exploration and exploitation. Exploration involves searching for points with high variance ($\sigma^2$), which corresponds to identifying regions of uncertainty. In contrast, exploitation involves selecting a value with a smaller predicted average ($\mu$) from previously searched points, with the aim of identifying a lower value around a known point. To achieve a good balance between exploration and exploitation, we introduce a lower confidence bound as the acquisition function. The lower confidence bound is calculated as the sum of the mean of the values and the confidence interval, and recommends the best place for further exploration. The formula for the lower confidence bound is as follows.

$$\begin{aligned} a(x;\lambda) &= \mu(x) - \kappa\sigma(x), \\ x_* &= \arg\min(a(x;\kappa)), \end{aligned} \tag{13}$$

where the $\kappa$ parameter denotes the sampling propensity of the model, whereby higher values of $\kappa$ correspond to an increase in the number of searches performed by the model. $\kappa$ is set to 1 in normal. $x_*$ denotes the subsequent point of sampling that is anticipated to yield a lower cost.

TABLE I
TESTCASES SPECIFICATIONS

| Bench | # of cells | Design time | Domain |
|---|---|---|---|
| aes | 12k | 4 $min$ | Source |
| keccak | 28k | 12 $min$ | Source |
| ldpc | 42k | 35 $min$ | Source |
| des3 | 68k | 20 $min$ | Source |
| nova | 144k | 40 $min$ | Target |
| rocket_chip | 691k | 180 $min$ | Target |

## IV. EXPERIMENTAL SETUP & RESULT

### A. Experimental Setup

The experiments were conducted on a computing system consisting of an AMD EPYC-Rome CPU with 15 cores operating at 2.8 GHz and 200 GB RAM, and an NVIDIA Geforce 1080 Ti graphics processing unit. The operating system used was CentOS Linux 7.9. The ASAP7 PDK [16] with BSIM-CMG model [17] was employed, with $OpenCores$ circuit designs [18] and $RocketChip$ [19] serving as test cases. The characteristics of the test cases are presented in TABLE I.

TABLE II
DESIGN & TECHNOLOGY PARAMETERS

| Parameter | min | max | Step size | # of combinations |
|---|---|---|---|---|
| LVT | 4.250 | 4.345 | 0.005 | 20 |
| RVT | 4.350 | 4.450 | 0.005 | 21 |
| VDD | 0.50 | 0.90 | 0.05 | 9 |
| TRACK | 6T | 7.5T | 1.5T | 2 |
| # of all combinations: 7,560 | | | | |

* LVT and RVT refer to the corresponding gate work function, respectively.
* Units for LVT and RVT are in $eV$, while VDD is in $V$.

### B. Process Design kits Generation

To generate process design kits (PDK), we modified the design and technology options described in Section II-A. To adjust the threshold voltage, we employed the BSIM-CMG model, which is capable of simulating device characteristics based on process changes. Specifically, we adjusted the gate work function parameter $phig$ to modify

TABLE III
PREDICTION: COMPARISON RESULTS OF PREDICTING POWER, AREA, AND PERFORMANCE (WNS)

| Metric | | Target 1 | | | | | | Target 2 | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | GP | STGP [10] | | | | MTGP | GP | STGP [10] | | | | MTGP |
| | | | ldpc | keccak | aes | des3 | | | ldpc | keccak | aes | des3 | |
| MAE | power | 655 | 532 | 775 | 532 | 491 | **386** | 29,513 | 16,108 | 21,535 | 26,232 | 16,046 | **13,691** |
| | area | 172 | 126 | 160 | 183 | 185 | **117** | 801 | 812 | 880 | 862 | 793 | **673** |
| | wns | 185.60 | 209.97 | 171.62 | 173.49 | 168.64 | **120.56** | **11.42** | 11.75 | 12.21 | 12.06 | 11.99 | 11.59 |
| R2 | power | 0.8673 | 0.9149 | 0.8693 | 0.9189 | 0.9279 | **0.9570** | 0.5789 | 0.9040 | 0.7839 | 0.6774 | 0.9015 | **0.9196** |
| | area | 0.9863 | 0.9922 | 0.9880 | 0.9843 | 0.9830 | **0.9935** | 0.9891 | 0.9841 | 0.9903 | 0.9899 | 0.9909 | **0.9934** |
| | wns | 0.3408 | 0.1821 | 0.4473 | 0.4309 | 0.4550 | **0.7334** | 0.5175 | 0.5431 | 0.4698 | 0.4842 | 0.4592 | **0.5749** |
| 95% CI ACC | power | 78.84% | 85.06% | 80.50% | 80.08% | 83.40% | **100.00%** | 65.98% | 80.91% | 70.54% | 64.73% | 78.84% | **96.68%** |
| | area | **100.00%** | 99.17% | 98.76% | **100.00%** | **100.00%** | **100.00%** | 99.17% | 98.76% | **100.00%** | **100.00%** | 99.17% | **100.00%** |
| | wns | 36.51% | 58.09% | 9.96% | 11.62% | 15.35% | **93.36%** | 70.12% | 16.18% | 52.28% | 55.60% | 60.17% | **83.40%** |

* The Target 1 is nova and the Target 2 is rocket_chip.
* The best results are shown in bold.
* 95% CI ACC measures the percentage of test data that fell within the 95% confidence interval.

the threshold voltage. Since modern designs employ Multi-$V_{th}$ technique to minimize leakage power, we utilized two $V_{th}$ cell libraries, LVT and RVT. In this context, LVT and RVT refer to low and regular threshold voltage cells respectively. Therefore, we specified each set of data points in the range of [4.250, 4.345] and [4.350, 4.450] in units of $eV$ as input parameters for a pair of NMOS gate work functions. The endpoints of this range corresponded to the values of the foundry-set SLVT and HVT, respectively, and the step length was set to 0.005 $eV$. In this context, the foundry-set SLVT and HVT refer to the super-low and high threshold voltage cell libraries provided by the foundry as standard, respectively. To ensure symmetric operation of CMOS, we first modified the NMOS gate work function, found the NMOS drain-source current $i_{ds}$ through SPICE simulation, and then adjusted the PMOS gate work function to match the PMOS $i_{ds}$ to 90% of the NMOS $i_{ds}$. We also considered the effect of supply voltage scaling using library characterization tools. We specified each data point in the range of [0.50, 0.90] with a step length of 0.05 $V$. Finally, we used 7.5T and 6T standard cells to represent high-performance and high-density cells, respectively, resulting in a total design space of 7,560. A summary of the input parameters is presented in TABLE II. Subsequently, logic synthesis was performed to assess the PPA of the modified design and technology options. The clock period for synthesis was determined as a value between the maximum clock periods that could be achieved using each foundry-set LVT and foundry-set RVT. For our experiments, *HSPICE v21.9* was used for SPICE simulation, *Silicon Smart v21.6* for library characterization, and *Design Compiler v18.6* for logic synthesis.

### C. Prediction Model Evaluation

In our study, we selected three evaluation metrics to assess the performance of our predictive model: mean absolute error (MAE), R2 score, and 95% confidence interval accuracy. Since GP predicts both the expected value ($\mu$) and the confidence interval ($\sigma^2$), we needed to include an evaluation indicator for the predicted confidence interval. To do this, we measured the accuracy of the test data within the 95% confidence interval predicted by the model. To evaluate the predictive model's performance, we utilized 250 grid data points that were evenly spaced in the design space. We used the ADAM [20] optimizer with 1,000 iterations to train each predictive model. To reduce learning time, multi-source transfer GP (MTGP) set $\alpha_{S,T}$ learned through single-source trasfer GP (STGP) as the initial value of MTGP, and then only learned 100 times. All of our models were implemented in *Python* using *PyTorch*.

We conducted an experiment to compare transfer performance using 200 of source design data and 10 of target design data. The experimental results are presented in the TABLE III. In our study, we compared the performance of MTGP and reference GP in predicting target design data. We found that MTGP outperformed reference GP by reducing both MAE of power and MAE of area by 47.33% and 24.07%, respectively, as well as reducing MAE of WNS by 16.75%. MTGP also showed improved R2 scores for power, area, and WNS, with the R2 score of WNS showing the most significant improvement at 63.14%. To illustrate the performance of our predictive model, we present the power prediction results for Target 2 as a representative example in Fig. 5. We further demonstrated that using multiple source design data for learning can increase the accuracy of the 95% confidence interval and prevent overfitting to a specific source design. These results suggest that building a predictive model using multiple sources is more effective than relying on data from a single source design.

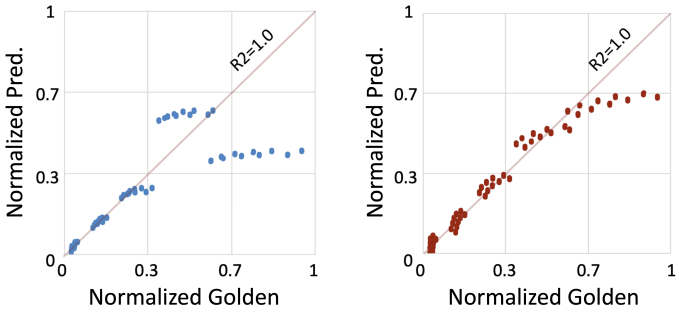


Fig. 5. Performance comparison of GP and MTGP for power prediction of Target 2: The power prediction plots in the left and right panels show the results obtained using the reference GP and MTGP models, respectively.

### D. Optimization Result

We utilized MTGP for DTCO optimization, building on the model obtained in Section IV-C. We performed optimization 30 times using Bayesian optimization, with $\alpha$, $\beta$, and $\gamma$ set to 1.0, 1.0, and 3.0, respectively. The reference design used the results using foundry-set LVT, foundry-set RVT, 0.70V and 7.5T. We conducted a Bayesian optimization on multiple GP models and analyzed their convergence performance. Fig. 6 shows the results, where the vertical axis represents cost, as per Equation (12), and the horizontal axis represents the number of optimization steps. To better capture trends, we plotted cost values as 11-step-sized moving averages. The plot clearly shows that MTGP outperforms other models in terms of optimization performance such as the convergence and stability for both targets.

The results presented in TABLE IV illustrate the minimum cost points that each model explored while meeting the timing constraints. These results demonstrate that Bayesian optimization using MTGP

TABLE IV
OPTIMIZATION: COMPARISON RESULTS OF OPTIMIZING POWER AND AREA WHILE MEETING TIMING CONSTRAINTS

| Metric | Target 1 | | | | | | Target 2 | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | GP | STGP [10] | | | | MTGP | GP | STGP [10] | | | | MTGP |
| | | ldpc | keccak | aes | des3 | | | ldpc | keccak | aes | des3 | |
| power | -28.10% | 30.36% | 2.68% | 28.91% | 3.15% | **34.06%** | 14.52% | 14.52% | 32.03% | 3.65% | 39.10% | **40.51%** |
| area | 0.83% | -0.86% | 0.45% | -0.72% | 0.08% | **17.80%** | -0.18% | -0.18% | **22.09%** | 22.07% | -0.27% | 21.93% |
| wns | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 |

* The Target 1 is nova and the Target 2 is rocket_chip.
* The best results are shown in bold.

outperforms conventional Bayesian optimization in terms of both optimization performance. Additionally, while the optimization results achieved through STGP can vary depending on the source and target design, MTGP consistently exhibits excellent performance across various conditions. For example, as shown in Fig. 6, STGP transferred $ldpc$ outperforms STGP transferred $keccak$ for Target 1, while the opposite is true for Target 2. As shown in Table III, the WNS's MAE of STGP transferred $ldpc$ at Target 1 was observed to be inferior to that of STGP transferred keccak, whereas the 95% confidence interval accuracy is significantly higher for STGP transferred $ldpc$. These observations can be explained by the fact that in Bayesian optimization, the selection of the next sampling point takes into account not only the predicted value but also the confidence interval, as per Equation (13). Hence, as MTGP outperforms STGP in terms of both MAE and accuracy of 95% confidence interval, it can be concluded that the optimization performance is robust and effective, without any overfitting issues. Our experimental results demonstrate that the proposed methodology improves power and area by an average of 37.29% and 19.87%, respectively, for both target designs compared to the reference design, while meeting the imposed timing constraints.

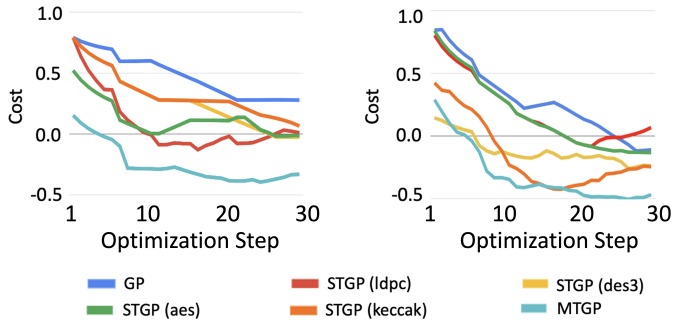


Fig. 6. Convergence comparison of GP, STGP, and MTGP. The left plot shows the convergence tendency for Target 1, and the right plot shows the convergence tendency for Target 2.

## V. CONCLUSION

In this study, we propose a DTCO framework based on the Multi-source Transfer Gaussian Process (MTGP) that effectively explores a vast design and technology space. Our approach transfers small design data to a larger design, improving the performance of the predictive model and demonstrating its scalability. Moreover, we introduce MTGP to integrate and transfer multiple source designs into the framework, achieving a lower generalization error than only transferring single source design. In our experiments, we achieved an average reduction of 47.33% and 24.07% in the average mean absolute error for power and area, respectively, compared to the reference GP. Furthermore, in the 7nm technology node, we achieved a average reduction of 37.29% and 19.87% in power and area, respectively, while meeting imposed timing constraints, compared to the reference design. Our experimental results demonstrate that our method reduces the optimization iterations required for DTCO and enables efficient design feedback.

## VI. ACKNOWLEDGMENTS

This work was partly supported by Institute for Information & communications Technology Promotion (IITP) grant funded by the Korea government (MSIT) (No.RS-2023-00222085, Development of memory module and memory compiler for non-volatile PIM optimized for data characteristics and data access characteristics of AI processor / No.2021-0-00754, Software Systems for AI Semiconductor Design / No.2022-0-01172, DRAM PIM Design Base Technology Development). Furthermore, this research was supported in part by Samsung Electronics Co., Ltd.

## REFERENCES

[1] T. Song, *et al.*, "3nm Gate-All-Around (GAA) Design-Technology Co-Optimization (DTCO) for succeeding PPA by Technology", *Proc. CICC*, 2022.
[2] V. Moroz, *et al.*, "DTCO Launches Moore's Law Over the Feature Scaling Wall", *Proc. IEDM*, 2020.
[3] CK, Cheng, *et al.*, "Complementary-FET (CFET) standard cell synthesis framework for design and system technology co-optimization using SMT", *IEEE Transactions on VLSI*, 2021, pp. 1178-1191.
[4] CK, Cheng, *et al.*, "PROBE2.0: A systematic framework for routability assessment from technology to design in advanced nodes", *IEEE Transactions on CAD*, 2021, pp. 1495-1508.
[5] J. Jeong, *et al.*, "A Study on Optimizing Pin Accessibility of Standard Cells in the Post-3 nm Node", *Proc. ISLPED*, 2022.
[6] B. Reagen, *et al.* "A case for efficient accelerator design space exploration via bayesian optimization", *Proc. ISLPED*, 2017.
[7] Y. Ma, *et al.*, "CAD tool design space exploration via Bayesian optimization", *Proc. MLCAD*, 2019.
[8] H. Geng, *et al.*, "PTPT: physical design tool parameter tuning via multi-objective Bayesian optimization", *IEEE TCAD*, 2022, pp. 178-189.
[9] J. Jung, et al, "METRICS2.1 and Flow Tuning in the IEEE CEDA Robust Design Flow and OpenROAD", *Proc. ICCAD*, 2021, pp. 1-9.
[10] H. Geng, *et al.*, "PPATuner: pareto-driven tool parameter auto-tuning in physical design via gaussian process transfer learning", *Proc. DAC*, 2022.
[11] Z. Zhang, *et al.*, "A fast parameter tuning framework via transfer learning and multi-objective bayesian optimization", *Proc. DAC*, 2022.
[12] V. De, *et al.*, "Near threshold voltage (NTV) computing: Computing in the dark silicon era", *IEEE Design & Test*, 2017, pp. 24-30.
[13] M. Mustafa, T. et al, "Threshold Voltage Sensitivity to Metal Gate Work-Function Based Performance Evaluation of Double-Gate n-FinFET Structures for LSTP Technology", *World Journal of Nano Science and Engineering*, 2013, pp. 17-22.
[14] P. Wei, *et al.* "Adaptive Transfer Kernel Learning for Transfer Gaussian Process Regression", *IEEE TPAMI*, 2022.
[15] P. Wei, *et al.*, "Transfer Kernel Learning for Multi-Source Transfer Gaussian Process Regression", *IEEE TPAMI*, 2022, pp. 10.1109. 2016.
[16] L. T. Clark, *et al.*, "ASAP7: A 7-nm finFET predictive process design kit", *Microelectronics Journal*, 2016, pp. 105-115.
[17] S. Khandelwal, *et al.*, "BSIM-CMG 110.0.0: Multi-gate MOSFET compact model: technical manual", 2014
[18] OpenCores: "Open Source IP-Cores", http://www.opencores.org.
[19] Asanovic, Krste, *et al.*, "The rocket chip generator", EECS Department, University of California, Berkeley, Tech., 2016.
[20] Kingma, D. P, *et al.*, "Adam: A method for stochastic optimization", arXiv preprint arXiv:1412.6980, 2014.